

Unit: 7

**FILE SYSTEM INTERFACE
MANAGEMENT**

Prepared by
Er. Binod Kumar Rajbhar

File Overview

File System:

- ❖ A file is a collection of correlated information which is recorded on secondary or non-volatile storage like magnetic disks, optical disks, and tapes.
- ❖ It is a method of data collection that is used as a medium for giving input and receiving output from that program.
- ❖ In general, a file is a sequence of bits, bytes, or records whose meaning is defined by the file creator and user.
- ❖ Every File has a logical location where they are located for storage and retrieval

Objective of File management System

- It provides I/O support for a variety of storage device types.
- Minimizes the chances of lost or destroyed data
- Helps OS to standardized I/O interface routines for user processes.
- It provides I/O support for multiple users in a multiuser systems environment.

File structure

- ❖ A File Structure needs to be predefined format in such a way that an operating system understands.
- ❖ It has an exclusively defined structure, which is based on its type.
- ❖ Three types of files structure in OS:
 - **A text file:** It is a series of characters that is organized in lines.
 - **An object file:** It is a series of bytes that is organized into blocks.
 - **A source file:** It is a series of functions and processes.

File Attributes:

- ❖ A file has a name and data.
- ❖ Moreover, it also stores Meta information like file creation date and time, current size, last modified date, etc.
- ❖ All this information is called the attributes of a file system.
- ❖ some important File attributes used in OS:
 - **Name:** It is the only information stored in a human-readable form.
 - **Identifier:** Every file is identified by a unique tag number within a file system known as an identifier.
 - **Location:** Points to file location on device.
 - **Type:** This attribute is required for systems that support various types of files.
 - **Size.** Attribute used to display the current file size.
 - **Protection.** This attribute assigns and controls the access rights of reading, writing, and executing the file.
 - **Time, date and security:** It is used for protection, security, and also used for monitoring

File Type

- ❖ It refers to the ability of the operating system to differentiate various types of files like text files, binary, and source files.
- ❖ However, Operating systems like MS_DOS and UNIX has the following type of files:

✓ Character Special File

- It is a hardware file that reads or writes data character by character, like mouse, printer, and more.

✓ Ordinary files

- These types of files stores user information.
- It may be text, executable programs, and databases.
- It allows the user to perform operations like add, delete, and modify.

✓ Directory Files

- Directory contains files and other related information about those files. Its basically a folder to hold and organize multiple files.

✓ Special Files

- These files are also called device files. It represents physical devices like printers, disks, networks, flash drive, etc.

Functions of File:

- Create file, find space on disk, and make an entry in the directory.
- Write to file, requires positioning within the file
- Read from file involves positioning within the file
- Delete directory entry, regain disk space.
- Reposition: move read/write position.

File Access Methods:

- ❖ File access is a process that determines the way that files are accessed and read into memory.
- ❖ Generally, a single access method is always supported by operating systems.
- ❖ Though there are some operating system which also supports multiple access methods.
- ❖ Three file access methods are:
 - . Sequential access
 - . Direct random access
 - . Index sequential access

✓ Sequential Access

- In this type of file access method, records are accessed in a certain pre-defined sequence.
- In the sequential access method, information stored in the file is also processed one by one.
- Most compilers access files using this access method.

✓ Random Access

- The random access method is also called direct random access.
- This method allow accessing the record directly.
- Each record has its own address on which can be directly accessed for reading and writing.

✓ Sequential Access:

- This type of accessing method is based on simple sequential access.
- In this access method, an index is built for every file, with a direct pointer to different memory blocks.
- In this method, the Index is searched sequentially, and its pointer can access the file directly.
- Multiple levels of indexing can be used to offer greater efficiency in access. It also reduces the time needed to access a single record.

File Operations

Some common file operations:

- File Create operation
- File Delete operation
- File Open operation
- File Close operation
- File Read operation
- File Write operation
- File Append operation
- File Seek operation
- File Get attribute operation
- File Set attribute operation
- File Rename operation

❖ File Create Operation:

- The file is created with no data.
- The file create operation is the first step of the file.
- Without creating any file, there is no any operation can be performed.

❖ File Delete Operation:

- File must has to be deleted when it is no longer needed just to free up the disk space.
- The file delete operation is the last step of the file.
- After deleting the file, it doesn't exist.

❖ **File Open Operation:**

- ✓ The process must open the file before using it.

❖ **File Close Operation:**

- The file must be closed to free up the internal table space, when all the accesses are finished and the attributes and the disk addresses are no longer needed.

❖ **File Read Operation:**

- The file read operation is performed just to read the data that are stored in the required file.

❖ **File Write Operation:**

- The file write operation is used to write the data to the file, again, generally at the current position.

❖ **File Append Operation:**

- The file append operation is same as the file write operation except that the file append operation only add the data at the end of the file.

❖ File Seek Operation:

- For random access files, a method is needed just to specify from where to take the data. Therefore, the file seek operation performs this task.

❖ File Get Attribute Operation:

- The file get attributes operation are performed by the processes when they need to read the file attributes to do their required work.

❖ File Set Attribute Operation:

- The file set attribute operation used to set some of the attributes (user settable attributes) after the file has been created.

❖ File Rename Operation:

-

File System Layout:

- ❖ A file system is a set of files, directories, and other structures.
- ❖ File systems maintain information and identify where a file or directory's data is located on the disk.
- ❖ In addition to files and directories, file systems contain a boot block, a superblock, bitmaps, and one or more allocation groups.
- ❖ An allocation group contains disk i-nodes and fragments.
- ❖ Each file system occupies one logical volume.

✓ Boot Block

- The boot block occupies the first 4096 bytes of the file system starting at byte offset 0 on the disk.
- The boot block is available to start the operating system.

✓ Superblock

- The superblock is 4096 bytes in size and starts at byte offset 4096 on the disk.
- The super- block maintains information about the entire file system and includes the following fields:
 - Size of the file system
 - Number of data blocks in the file system
 - A flag indicating the state of the file system
 - Allocation group sizes

✓ Allocation Bitmaps:

- The file system contains two allocation bitmaps:
- The fragment allocation map records the allocation state of each fragment.
- The disk i-node allocation map records the status of each i-node.

✓ Fragments :

- Many file systems have disk blocks or data blocks.
- These blocks divide the disk into units of equal size to store the data in a file or directory's logical blocks.
- The disk block may be further divided into fixed-size allocation units called fragments

✓ Disk I-Nodes:

- A logical block contains a file or directory's data in units of 4096 bytes.
- Each logical block is allocated fragments for the storage of its data.
- Each file and directory has an i-node that contains access information such as file type, access permissions, owner's ID, and number of links to that file.
- These i-nodes also contain "addresses" for finding the location on the disk where the data for a logical block is stored.
- Each i-node has an array of numbered sections.
- Each section contains an address for one of the file or directory's logical blocks.

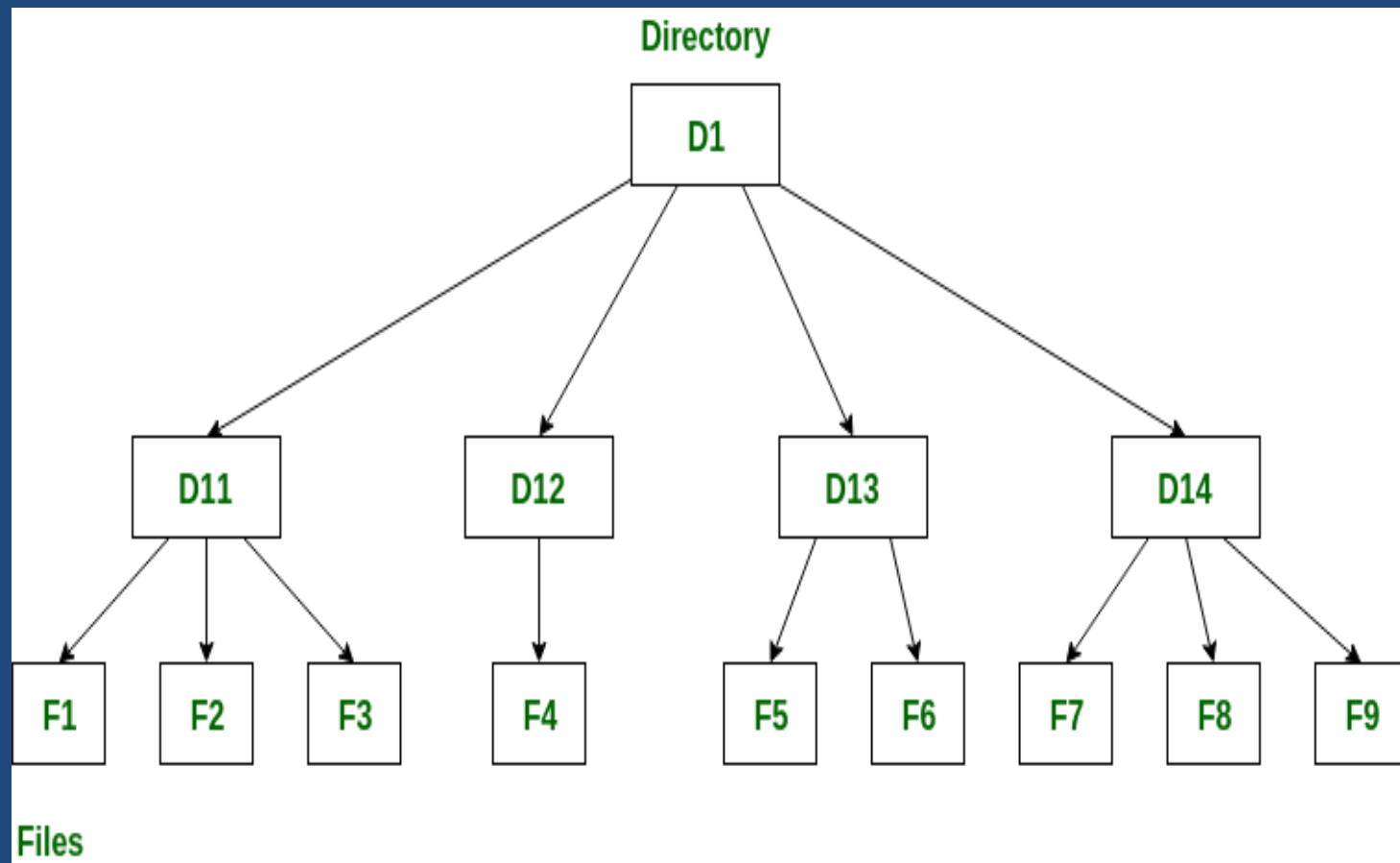
✓ Allocation Groups:

- The set of fragments making up the file system are divided into one or more fixed-sized units of contiguous fragments.
- Each unit is an allocation group.
- The first of these groups begins the file system and contains a reserved area occupying the first 32 x 4096 bytes of the group.
- The first 4096 bytes of this area hold the boot block and the second 4096 bytes hold the file system superblock.

Directory Structure

Introduction:

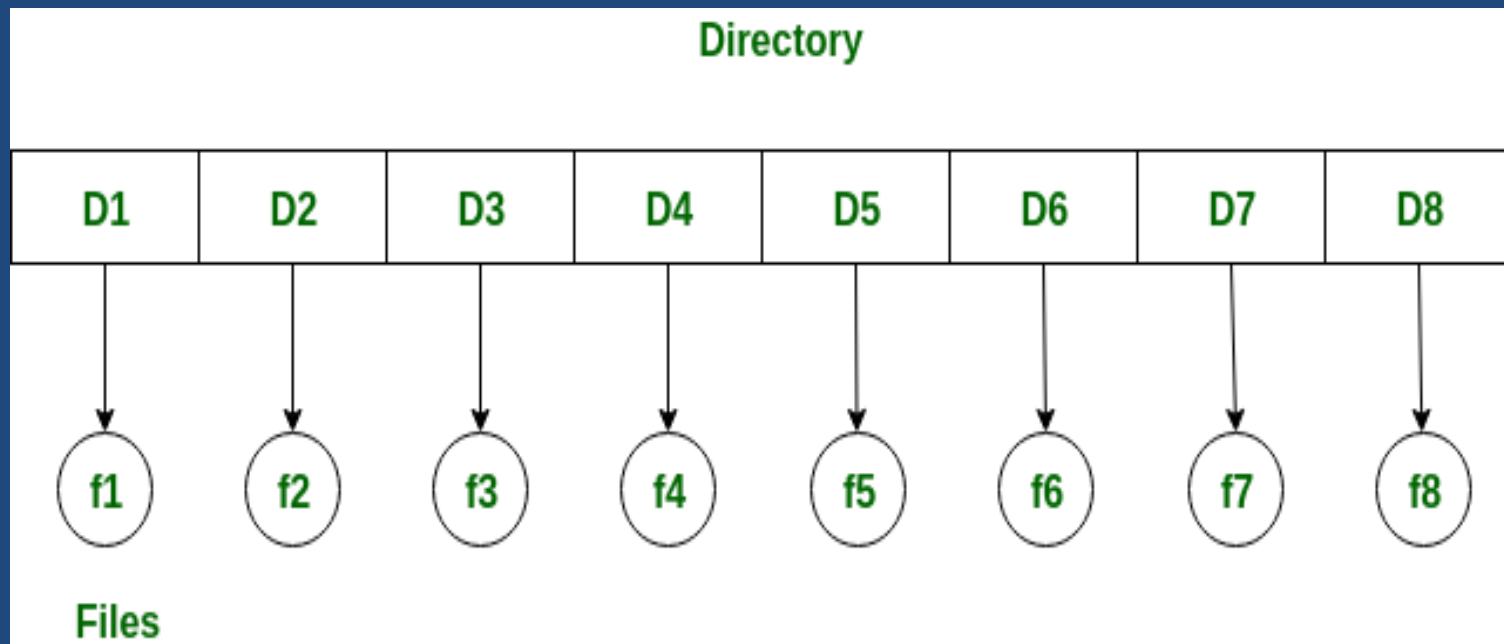
- ❖ A directory is a node containing information about files, it is also called folder.
- ❖ Directory can be defined as the listing of the related files on the disk.
- ❖ The directory may store some or the entire file attributes .
- ❖ A **directory** is a container that is used to contain folders and file.
- ❖ It organizes files and folders into a hierarchical manner.



There are several logical structures of a directory, these are given below.

❖ Single-level directory –

- ✓ Single level directory is simplest directory structure.
- ✓ In it all files are contained in same directory which make it easy to support and understand.
- ✓ A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user.
- ✓ Since all the files are in the same directory, they must have the unique name.
- ✓ If two users call their dataset test, then the unique name rule violated.



Advantages:

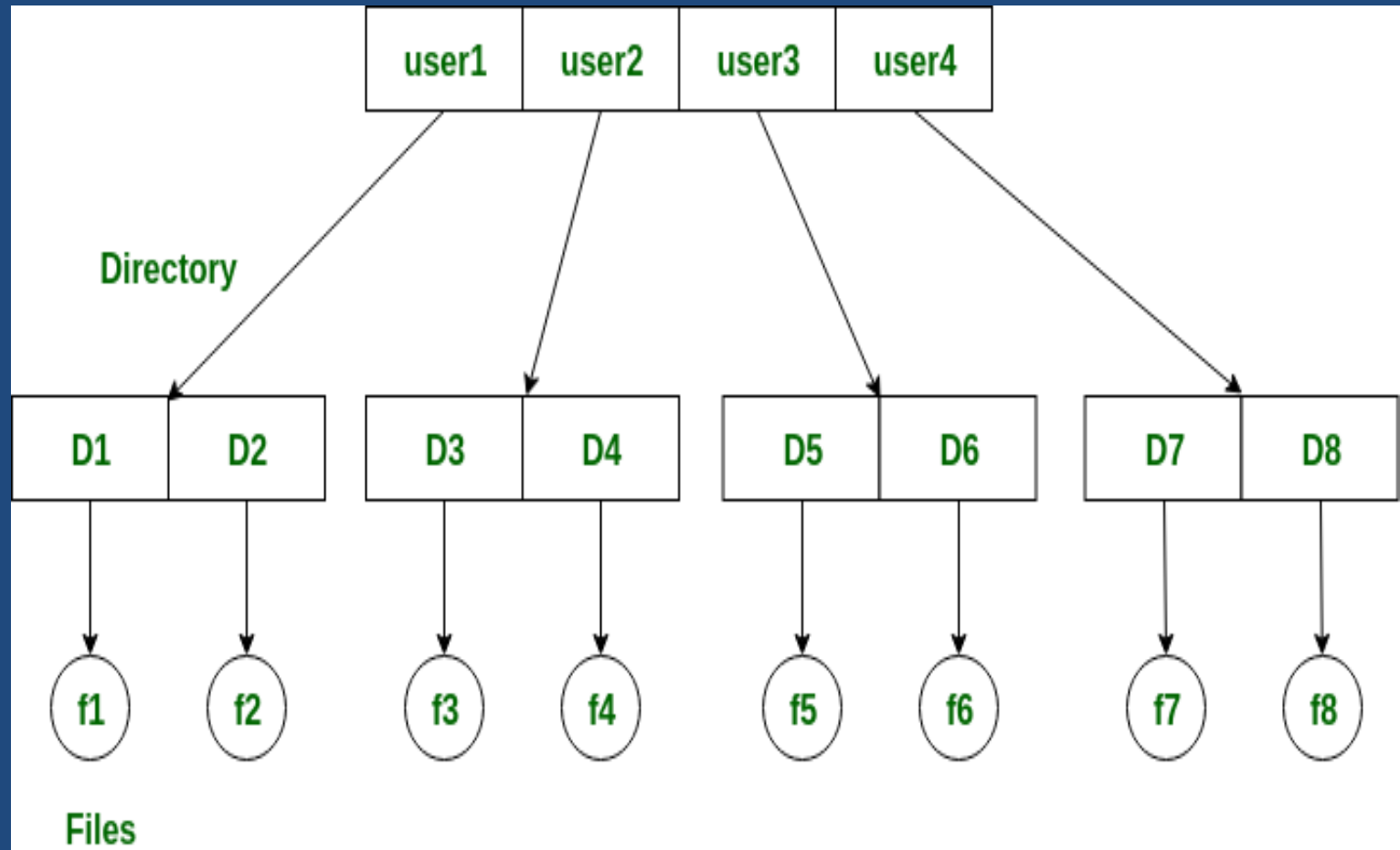
- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

Disadvantages:

- There may chance of name collision because two files can not have the same name.
- Searching will become time taking if the directory is large.
- In this can not group the same type of files together.

❖ Two-level directory –

- As we have seen, a single level directory often leads to confusion of files names among different users.
- The solution to this problem is to create a separate directory for each user.
- In the two-level directory structure, each user has their own *user files directory (UFD)*.
- The UFDs has similar structures, but each lists only the files of a single user.
- System's *master file directory (MFD)* is searched whenever a new user id=s logged in.
- The MFD is indexed by username or account number, and each entry points to the UFD for that user.



Advantages:

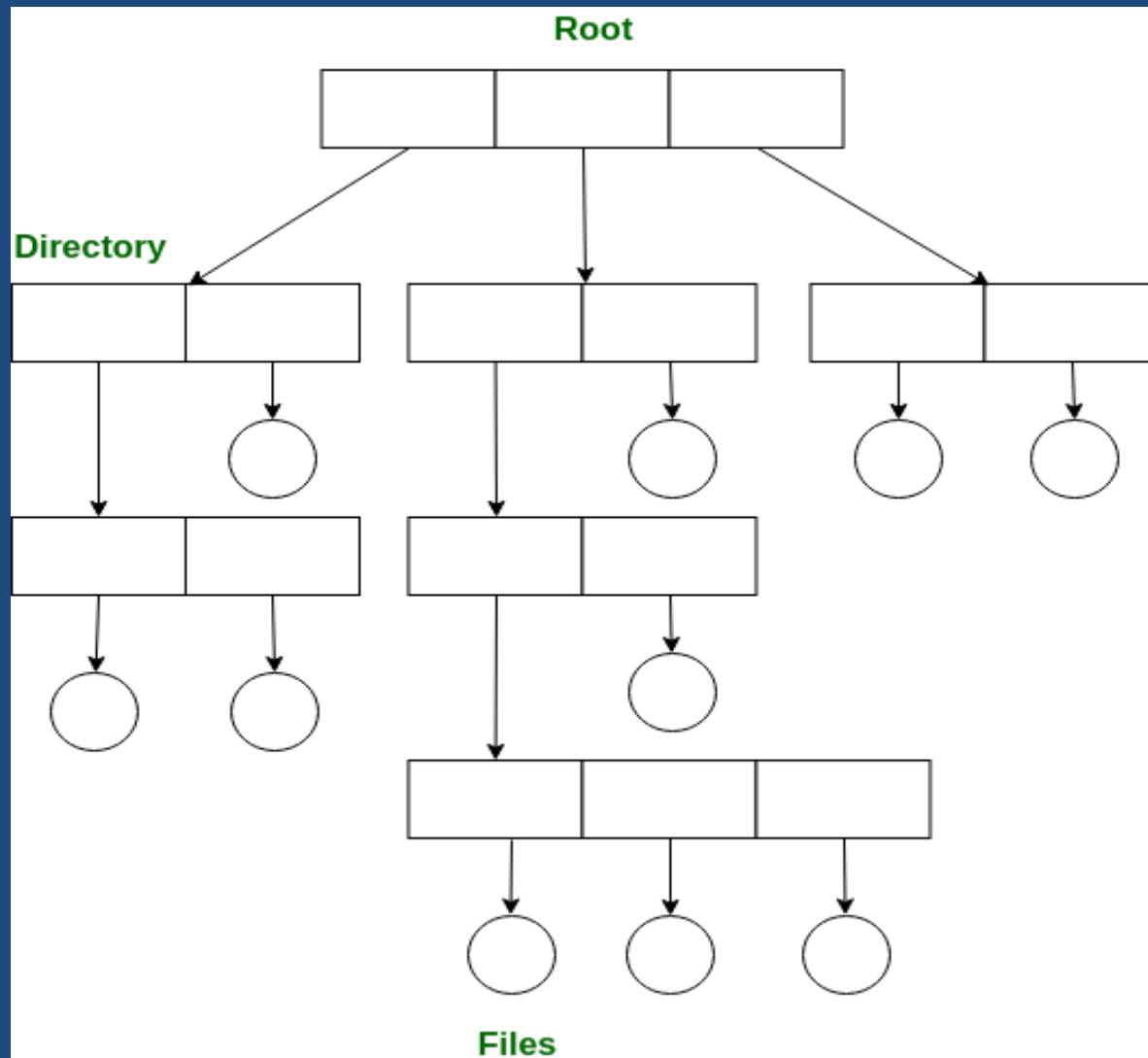
- We can give full path like /User-name/directory-name/.
- Different users can have same directory as well as file name.
- Searching of files become more easy due to path name and user-grouping.

Disadvantages:

- A user is not allowed to share files with other users.
- Still it not very scalable, two files of the same type cannot be grouped together in the same user.

Tree-structured directory

- ✓ Once we have seen a two-level directory as a tree of height 2, the natural generalization is to extend the directory structure to a tree of arbitrary height.
- ✓ This generalization allows the user to create their own subdirectories and to organize on their files accordingly
- ✓ A tree structure is the most common directory structure.
- ✓ The tree has a root directory, and every file in the system have a unique path.



Advantages:

- Very generalize, since full path name can be given.
- Very scalable, the probability of name collision is less.
- Searching becomes very easy, we can use both absolute path as well as relative.

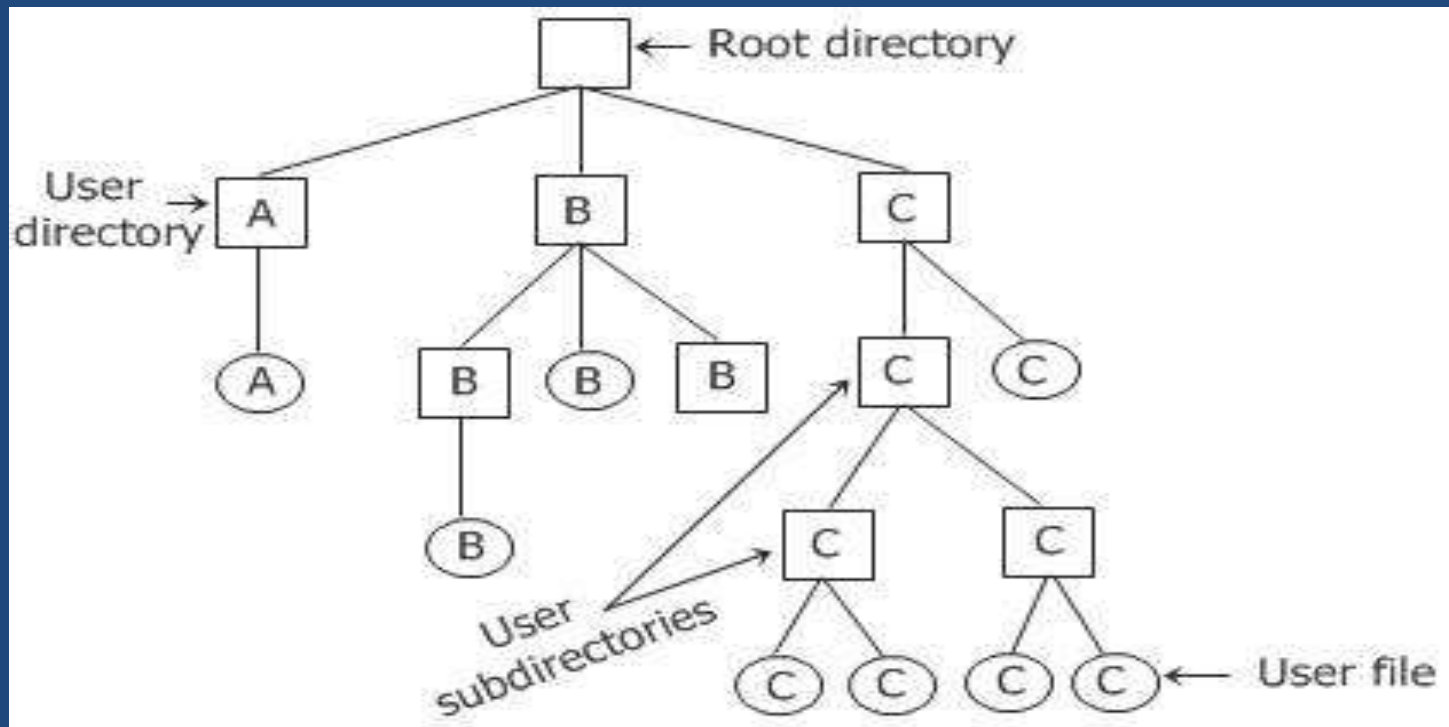
Disadvantages:

- Every file does not fit into the hierarchical model, files may be saved into multiple directories.
- We cannot share files.
- It is inefficient, because accessing a file may go under multiple directories.

❖ Hierarchical Directory Systems:

- Hierarchical directory systems are used for users with a large number of files, as the single-level directory system and two-level directory system is not satisfactory.
- The two-level directory system even on a single-user PC is inconvenient.
- Since it is common for the users, want to group their files together in logical ways.
- With hierarchical approach, each user on the computer system can have as many directories as are needed, so that files can be grouped together in natural ways.

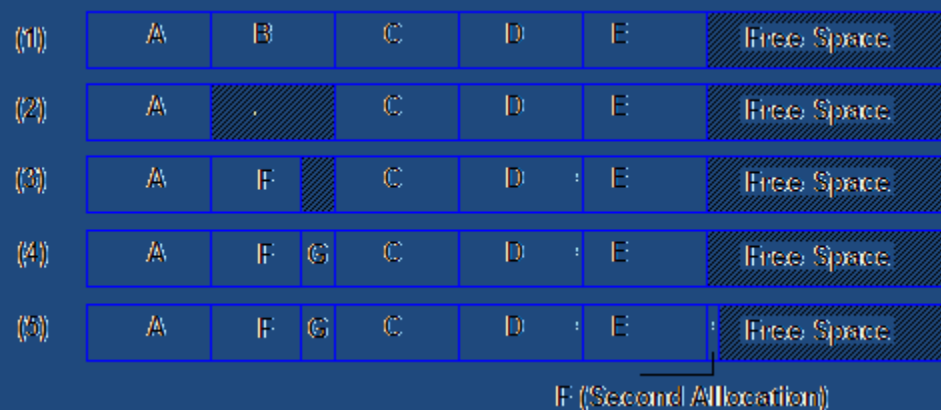
The figure given below shows this hierarchical directory system:



- As you can see from the above figure, the directories X, Y, and Z, contained in the root directory where each belong to a different user, two of them have created the subdirectories for the project they are working on.

File System Implementation:

- ❖ File system implementation defines how files and directories are stored, how disk space is managed, and how to make everything work efficiently and reliably.

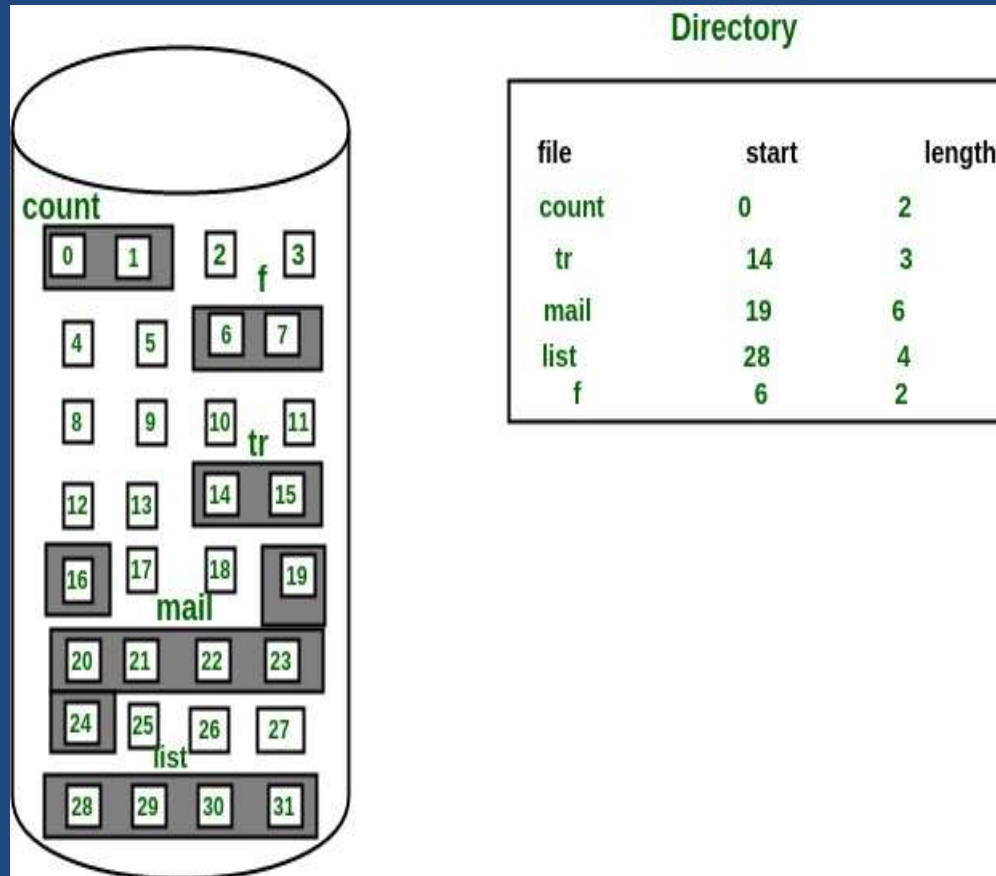


File Allocation Methods:

- ❖ The allocation methods define how the files are stored in the disk blocks.
- ❖ There are three main disk space or file allocation methods.
 - Contiguous Allocation
 - Linked Allocation
 - Indexed Allocation
- ❖ The main idea behind these methods is to provide:
 - Efficient disk space utilization.
 - Fast access to the file blocks.

1. Contiguous Allocation

- ❖ In this scheme, each file occupies a contiguous set of blocks on the disk.
- ❖ For example, if a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$.
- ❖ This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file.
- ❖
 - The directory entry for a file with contiguous allocation contains
 - Address of starting block
 - Length of the allocated portion.
 - ❖ The file '*mail*' in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks.



- **Advantages:**

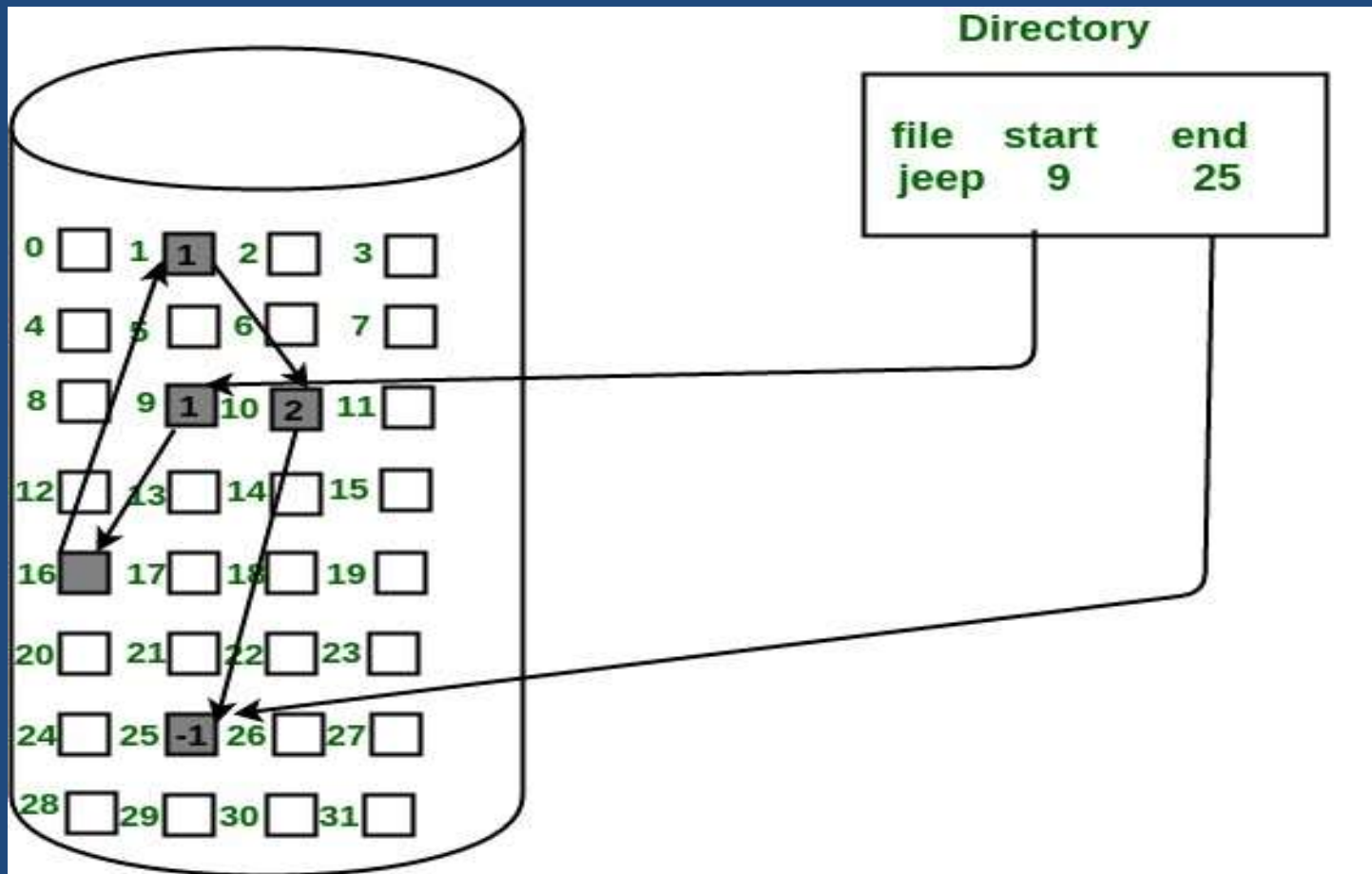
- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b+k)$.
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

- **Disadvantages:**

- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

2. Linked List Allocation

- ❖ In this scheme, each file is a linked list of disk blocks which **need not be** contiguous.
- ❖ The disk blocks can be scattered anywhere on the disk.
- ❖ The directory entry contains a pointer to the starting and the ending file block. Each block contains a pointer to the next block occupied by the file.
- *The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.*



Advantages:

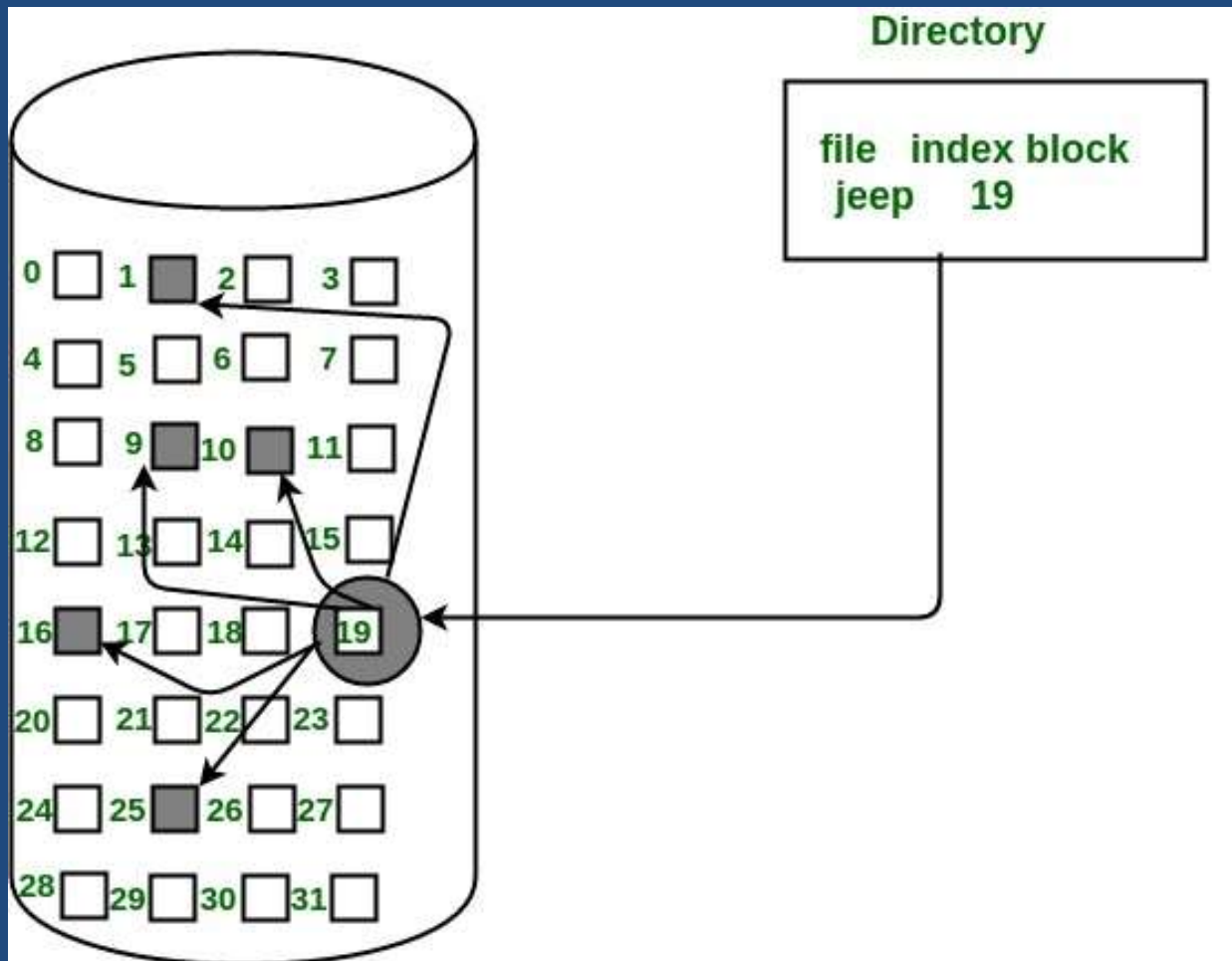
- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages:

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

3. Indexed Allocation

- ❖ In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file.
- ❖ Each file has its own index block.
- ❖ The i th entry in the index block contains the disk address of the i th file block.
- ❖ The directory entry contains the address of the index block as shown in the image:



Advantages:

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

Protection

- **Access Control list (ACL):**

- ✓ An access control list (ACL) is a list of rules that specifies which users or systems are granted or denied access to a particular object or system resource.
- ✓ Access control lists are also installed in routers or switches, where they act as filters, managing which traffic can access the network.
- ✓ Each system resource has a security attribute that identifies its access control list.
- ✓ The list includes an entry for every user who can access the system.
- ✓ The most common privileges for a file system ACL include the ability to read a file or all the files in a directory, to write to the file or files, and to execute the file if it is an executable file or program.

What are access control lists used for?

- ✓ Access control lists are used for controlling permissions to a computer system or computer network.
- ✓ They are used to filter traffic in and out of a specific device.
- ✓ Those devices can be network devices that act as network gateways or endpoint devices that users access directly.
- ✓ Access control lists can help organize traffic to improve network efficiency and to give network administrators granular control over users on their computer systems and networks.
- ✓ ACLs can also be used to improve network security by keeping out malicious traffic.

How do ACLs work?

Each ACL has one or more access control entries (ACEs) consisting of the name of a user or group of users. The user can also be a role name, such as *programmer* or *tester*. For each of these users, groups or roles, the access privileges are stated in a string of bits called an *access mask*. Generally, the system administrator or the object owner creates the access control list for an object.

Types of access control lists

There are two basic types of ACLs:

- **File system ACLs** manage access to files and directories. They give OSes the instructions that establish user access permissions for the system and their privileges once the system has been accessed.
- **Networking ACLs** manage network access by providing instructions to network switches and routers that specify the types of traffic that are allowed to interface with the network.

ACLs can also be categorized by the way they identify traffic:

- ✓ **Standard ACLs** block or allow an entire protocol suite using source IP addresses.
- ✓ **Extended ACLs** block or allow network traffic based on a more differentiated set of characteristics that includes source and destination IP addresses and port numbers, as opposed to just source address.

Access Control Matrix

Access Control Matrix

An **access control matrix** is a table that defines access permissions between specific subjects and objects. A matrix is a data structure that acts as a table lookup for the operating system. For example, Table 4.1 is a matrix that has specific access permissions defined by user and detailing what actions they can enact. User rdeckard has read/write access to the data file as well as access to the data creation application. User etyrell can read the data file and still has access to the application. User rbatty has no access within this data access matrix.

Table 4.1. User Access Permissions

Users	Data Access File # 1	Data Creation Application
rdeckard	Read/Write	Execute
etyrell	Read	Execute
rbatty	None	None

The rows of Table 4.1 show the capabilities of each subject; each row is called a capability list. The columns of Table 4.1 show the ACL for each object or application.